

任务 6: MyBatis-Plus 分页查询

在真实的企级项目中，用户数据往往成千上万，一次性查询全部数据（`selectList(null)`）会撑爆服务器内存；同时，业务数据往往是相互拆分的，我们需要跨越多张表来组合获取完整数据。

本任务将实现**分页查询**，下一次任务实现**自定义多表联查**。

1. 准备多表关联的数据库结构

为了演示多表联查和分页，我们需要在数据库中再建一张表。通常为了保证核心登录表（`sys_user`）的轻量化，我们会将用户的扩展信息存放在一张独立的“用户个人信息表”（`user_info`）中，它们之间构成**一对一**的关系。

请打开 PgAdmin，在数据库中执行以下 SQL 语句：

SQL

```
-- 1. 创建 user_info 用户个人信息表
CREATE TABLE user_info (
    id SERIAL PRIMARY KEY,
    real_name VARCHAR(50),      -- 真实姓名
    phone VARCHAR(20),          -- 手机号码
    address VARCHAR(200),       -- 联系地址
    user_id INT NOT NULL UNIQUE -- 关联 sys_user 表的 id。UNIQUE 保证一个账号只能
);

-- 2. 插入几条测试数据（sys_user 表中插入 id 为 1 和 2 的用户）
INSERT INTO user_info (real_name, phone, address, user_id) VALUES ('张三', '1380', '北京市海淀区', 1);
INSERT INTO user_info (real_name, phone, address, user_id) VALUES ('李四', '1390', '北京市海淀区', 2);
```

2. 开启 MyBatis-Plus 分页插件

MyBatis-Plus 的分页必须在 Spring Boot 中注册它的分页拦截器 (Interceptor)，否则分页代码会失效，退化为查全部。

在工程中 `config` 包下创建 `MyBatisPlusConfig` 配置类：

Java

```
import com.baomidou.mybatisplus.annotation.DbType;
import com.baomidou.mybatisplus.extension.plugins.MybatisPlusInterceptor;
import com.baomidou.mybatisplus.extension.plugins.inner.PaginationInnerInterceptor;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration // 标识这是一个配置类，Spring 启动时会自动加载
public class MyBatisPlusConfig {

    @Bean
    public MybatisPlusInterceptor mybatisPlusInterceptor() {
        MybatisPlusInterceptor interceptor = new MybatisPlusInterceptor();
        // 添加分页插件，指定底层数据库类型为 PostgreSQL
        interceptor.addInnerInterceptor(new PaginationInnerInterceptor(DbType.P
        return interceptor;
    }
}
```

3. 实现分页查询 (查询核心用户列表)

重构 `UserController` 和 `UserService`，新增一个按页码获取用户列表的接口。

3.1 UserService 接口与实现

在 `UserService` 接口中追加方法：

Java

```
// 获取用户分页数据
Result<Object> getUserPage(Integer pageNum, Integer pageSize);
```

在 `UserServiceImpl` 中实现该方法：

Java

```

import com.baomidou.mybatisplus.extension.plugins.pagination.Page;

@Override
public Result<Object> getUserPage(Integer pageNum, Integer pageSize) {
    // 1. 创建分页对象 (参数1: 当前页码, 参数2: 每页显示条数)
    Page<User> pageParam = new Page<>(pageNum, pageSize);

    // 2. 执行分页查询 (参数1: 分页对象, 参数2: 查询条件 Wrapper, 这里传 null 代表
    // 框架会自动执行一条 COUNT 语句查总数, 再拼接 LIMIT 执行分页
    Page<User> resultPage = userMapper.selectPage(pageParam, null);

    // 3. 返回结果 (resultPage 中包含了 records 数据列表、total 总条数、pages 总页
    return Result.success(resultPage);
}

```

3.2 UserController 新增接口

在 `UserController` 中暴露分页查询的 HTTP 接口:

Java

```

// 分页查询用户列表 - 路径为 GET /api/users/page
@GetMapping("/page")
public Result<Object> getUserPage(
    @RequestParam(defaultValue = "1") Integer pageNum,
    @RequestParam(defaultValue = "5") Integer pageSize) {
    return userService.getUserPage(pageNum, pageSize);
}

```

附加题 1 : Postman 分页接口联调验证

重启项目, 打开 Postman 进行验证。

1. 测试分页接口:

- Method: GET

- **URL:** `http://localhost:8080/api/users/page?pageNum=1&pageSize=2`
- **期望结果:** 返回的 JSON 数据中包含了 `records` (当前页的数据数组)、`total` (总条数)、`current` (当前页码) 等完整的元数据。

2. 验证物理分页日志

提交要求

1. **测试与截图:** 请截取 “Postman 分页查询用户的返回 JSON 结果图”。
2. **文档提交:** 完成任务 6 后, 将新增的 `MyBatisPlusConfig` 及核心改动推送至个人仓库分支。将**任务成功截图** 粘贴至作业 Word 文档中, 命名为《学号_姓名》.docx/.doc, 并按时提交至慧学平台。